

Message Generation Under Structural Constraints Using LLMs

Dario Francisco Alfonso Urrutia

Universidad de La Habana, La Habana, Cuba

Abstract. This paper presents a system for generating messages that satisfy a set of structural constraints, using a large language model (LLM) as both generator and evaluator. Given a topic and a collection of constraints — such as word count limits, required words, and forbidden words — the system employs a greedy optimization algorithm that iteratively generates candidate messages and retains the best result found.

The system was evaluated on a dataset of 10 benchmark instances across three difficulty levels. Two evaluator configurations were compared: a deterministic structural evaluator and an LLM-based semantic evaluator. The structural evaluator achieved a perfect constraint satisfaction rate on all instances in a single iteration, while the LLM-based evaluator required all available iterations and failed on one hard instance due to the generator ignoring prompt instructions.

Results show that deterministic evaluation is more reliable and significantly faster — approximately 8 times — than LLM-based evaluation for structural constraints. These findings suggest that LLMs are best used as generators in this setting, while deterministic evaluation should be preferred for exact constraint checking.

Keywords: LLM · constraint satisfaction · text generation · greedy optimization

1 Introduction

Generating text that satisfies a set of structural and semantic constraints is a challenging problem. The space of possible messages is infinite, and semantic constraints cannot be evaluated with deterministic functions. Classical constraint satisfaction approaches require a finite, discrete domain, which makes them unsuitable for natural language generation.

Large language models (LLMs) offer a promising solution: they can generate fluent text and evaluate semantic properties through natural language reasoning. In this work, we present a system that uses an LLM in two roles: as a *generator* that produces candidate messages, and as an *evaluator* that scores how well each candidate satisfies semantic constraints.

The system frames message generation as an optimization problem. Given a topic and a set of constraints, a greedy optimizer iteratively generates candidate messages and evaluates them, retaining the best result found.

The source code is publicly available at <https://github.com/D4R102004/llm-constrained-text-generator>.

2 Problem Description and Formal Model

Given a topic t and a set of constraints $C = \{c_1, c_2, \dots, c_n\}$, the goal is to find a message m^* that maximizes the number of satisfied constraints:

$$m^* = \arg \max_{m \in \mathcal{M}} \frac{1}{n} \sum_{i=1}^n c_i(m) \quad (1)$$

where $\mathcal{M}$ is the space of all possible messages and $c_i(m) \in \{0, 1\}$ indicates whether message m satisfies constraint c_i . The normalized sum defines the Constraint Satisfaction Rate (CSR).

Constraints are divided into two categories:

- **Structural constraints:** deterministic rules that can be evaluated programmatically (e.g., word count limits, required or forbidden words).
- **Semantic constraints:** properties that require natural language understanding to evaluate (e.g., tone, topic relevance).

3 Dataset

The dataset consists of 10 benchmark instances generated manually for this work. Each instance defines a topic and a set of constraints that the generated message must satisfy. Instances are organized into three difficulty levels based on the number and complexity of constraints:

- **Easy** (3 instances): 2 constraints — minimum word count and one required word.
- **Medium** (4 instances): 3 constraints — minimum and maximum word count, one required word, and optionally one forbidden word.
- **Hard** (3 instances): 4 constraints — minimum and maximum word count, multiple required words, and one forbidden word.

Topics were selected to be generic and domain-independent, ensuring that the LLM can generate fluent text without specialized knowledge. Constraints were chosen from the four types implemented in the system: `MinWordsConstraint`, `MaxWordsConstraint`, `RequiredWordConstraint`, and `ForbiddenWordConstraint`.

The dataset was designed to provide a range of difficulty levels suitable for behavioral analysis of the system.

4 Algorithm Design

The system uses a greedy optimization strategy implemented in the `GreedyOptimizer` class. The algorithm iteratively generates candidate messages and retains the best result found so far.

The optimization loop proceeds as follows:

1. Initialize the best message as an empty string with score 0.0.
2. At each iteration, identify the constraints that the current best message fails to satisfy.
3. Pass the failing constraints as feedback to the generator, which uses them to guide the next generation attempt.
4. Evaluate the new candidate message against all constraints.
5. If the new score exceeds the current best, update the best message.
6. Terminate early if a perfect score ($\text{CSR} = 1.0$) is achieved, or stop after the maximum number of iterations.

The algorithm is greedy in the sense that it always retains the highest-scoring candidate found so far, without backtracking. The feedback mechanism allows the generator to focus on the constraints that failed in previous attempts, guiding the search toward better solutions.

5 Role of the LLM

The system uses a large language model (LLM) in two distinct roles:

Generator. The LLM receives a prompt containing the topic, the list of constraints described in natural language, and optionally a list of constraints that failed in previous attempts. It generates a candidate message as a plain text response.

Evaluator. The LLM acts as a judge for semantic constraints. For each constraint, it receives the candidate message and the constraint description, and returns a score between 0.0 and 1.0 indicating how well the message satisfies the constraint.

Structural constraints (word count, required and forbidden words) are evaluated deterministically by Python code, not by the LLM. This design decision ensures that structural evaluation is exact and reproducible, since LLMs can produce inconsistent results when applied to deterministic checks.

All LLM interactions use the `llama3.2:3b` model served locally via Ollama at `http://localhost:11434`.

6 Experimental Methodology

Experiments were conducted to compare two evaluator configurations:

- **Structural:** uses `StructuralEvaluator`, which computes the CSR deterministically by checking each constraint with Python code.

- **Ollama**: uses `OllamaEvaluator`, which queries the LLM to score each constraint independently.

Both configurations use the same `OllamaGenerator` and `GreedyOptimizer`. The independent variable is the evaluator used to guide the optimization. The dependent variable is the final CSR, always computed by `StructuralEvaluator` to ensure a comparable, objective metric across both configurations.

For each instance and evaluator configuration, the optimizer was executed and the following metrics were recorded: final CSR, number of iterations used, elapsed time, score progression across iterations, and per-constraint satisfaction status for each constraint.

7 Results and Analysis

Table 1 summarizes the final CSR and the elapsed time for each instance and evaluator configuration.

Table 1. Experimental results by instance and evaluator.

Instance	Difficulty	Str. CSR	Oll. CSR	Str. time (s)	Oll. time (s)
ai_education	easy	1.00	1.00	157	1063
healthy_habits	easy	1.00	1.00	117	932
renewable_energy	easy	1.00	1.00	191	1200
remote_work	medium	1.00	1.00	168	1415
cybersecurity	medium	1.00	1.00	172	1378
climate_change	medium	1.00	1.00	177	1470
online_learning	medium	1.00	1.00	152	1337
space_exploration	hard	1.00	1.00	198	1733
mental_health	hard	1.00	0.50	201	2515
sustainable_cities	hard	1.00	1.00	184	2454

Three main findings emerged from the experiments.

First, `StructuralEvaluator` achieved $\text{CSR} = 1.0$ on all 10 instances in a single iteration. This is because structural constraints are deterministic and the LLM generator is generally capable of following explicit instructions such as word count limits and required words.

Second, `OllamaEvaluator` always used all 3 iterations without converging early. This suggests that the LLM evaluator provides noisy signals — it does not consistently recognize when constraints are satisfied, so the optimizer never triggers the early exit condition.

Third, `OllamaEvaluator` failed on the `mental_health` instance with $\text{CSR} = 0.50$. Inspection of the generated message revealed that the LLM ignored the prompt instruction to respond with the message only, and instead produced an explanation of the rules alongside the message. This caused the word count to

exceed the maximum limit and introduced a forbidden word in the explanation text.

Regarding efficiency, `StructuralEvaluator` completed all 10 instances in approximately 30 minutes, while `OllamaEvaluator` required approximately 4 hours — roughly 8 times slower — due to the additional LLM calls required for semantic evaluation.

8 Limitations and Future Work

The current system has two main limitations.

First, the evaluation phase is slow when using `OllamaEvaluator`, since it performs one LLM call per constraint per iteration. For an instance with four constraints and three iterations, this results in twelve evaluation calls. This does not scale well to larger datasets or higher iteration counts.

Second, the greedy optimizer does not guarantee convergence to a perfect solution. With a limited number of iterations, the system may terminate before finding a message that satisfies all constraints, returning the best candidate found so far.

Future improvements could include evaluating constraints in parallel to reduce latency, and replacing the greedy strategy with a more sophisticated search algorithm such as beam search, which maintains multiple candidate messages simultaneously.